

Reviewer Pack — Dilworth Width of Gate-Cone Poset Lower-Bounds AC^0 PARITY Si...

Entry f2533eda1d01 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:31:02 UTC

Dilworth Width of Gate-Cone Poset Lower-Bounds AC^0 PARITY Size

Entry ID: f2533eda1d01 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 02:31:02 UTC

1. Conjecture

Field A (mathematical branch): Order theory — Dilworth’s theorem and König-matching computation of poset width on Boolean-lattice containment orders (Dilworth 1950 ‘A decomposition theorem for partially ordered sets’; Mirsky 1971; Fulkerson 1956 ‘Note on Dilworth’s decomposition theorem’; Trotter 1992 ‘Combinatorics and Partially Ordered Sets’). A non-ring set-inclusion functional: width via Hopcroft–Karp matching on the strict-comparability bipartite graph uses only Boolean subset tests and integer matching, with NO polynomial F_q extension (containment of supports in 2^1 is not preserved under polynomial-ring lifts of the Boolean ring), so the bridge is Aaronson–Wigderson algebrization-safe. **Field B** (complexity object): AC^0 depth- d circuit size $s(C)$ for the explicit n -bit PARITY function, in the Håstad 1986 switching-lemma / Razborov–Smolensky / Beame 1994 regime; the canonical $2^{\Theta(n \{1/(d-1)\})}$ target and the $V^0 \sqsubseteq V^0(\oplus)$ bounded-arithmetic stepping stone.

Statement:

Let C be an AC^0 circuit of depth d and size $s = |V(C)|$ with $C \equiv \text{PARITY}_n$, and define the gate-cone poset $P(C)$ on $V(C)$ by $g \sqsubseteq h$ iff $\text{support}(g) \subseteq \text{support}(h)$ where $\text{support}(g) := \{x \in \{0,1\}^n : g(x)=1\}$; let $w(C) := \text{width}(P(C))$ be Dilworth’s maximum-antichain size, computed by König matching as $|V(C)| - \mu(B(C))$ on the strict-comparability bipartite graph $B(C)$. Conjecture: there exists an absolute constant $c > 0$ such that $w(C) \geq c \cdot \log_2(s)$ for every AC^0 circuit C computing PARITY_n ; equivalently $s \leq 2^{\{w(C)/c\}}$. A single AC^0 instance $C \equiv \text{PARITY}_n$ with $w(C)/\log_2(s) < c$ for every fixed c refutes it.

Rationale (proposer’s reasoning):

A pairwise-incomparable family of gate cones is a ‘structural fooling set’ in the containment lattice — antichains here play the role of rectangle-discrepancy fooling sets that LIFTING transfers from query (decision-tree certificates) to communication, but for SIZE rather than depth, since min chain cover (Dilworth dual) equals the minimum number of monotone monocoloured chains needed to schedule the gate evaluation. The invariant is structural on the circuit wiring — two AC^0 circuits with identical truth table (e.g. one with redundant subsumed gates) can have very different widths, shielding from Razborov–Rudich natural proofs — and the Boolean-lattice subset relation does not algebrize.

Taxonomy category: LIFTING (status at proposal time: alive)

¹_n

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9e318e58042e9af2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 360 AC⁰ PARITY instances ($n \in \{8, 10, 12, 14\} \times 30$ seeds $\times$ 3 constructions {C2_DNF, C2_CNF, C4_blockXOR} with random subsumed padding), compute ratio $r(C) = w(C) / \log_2(\text{size}(C))$ via Hopcroft-Karp on $B(C)$; conjecture is SUPPORTED iff $\min_{\text{instance}} r(C) \geq 0.4$ AND $\text{mean } r(C) \geq 0.5$; FALSIFIED iff any single instance yields $r(C) < 0.4$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.86	SAFE	SAFE
NATURAL PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Dilworth width Boolean circuit lower bound antichain AC⁰ PARITY-gate poset support containment circuit complexity switching lemma-partial order width bounded depth circuit size lower bound parity

Top relevant hits considered: - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC⁰ and TC⁰ - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/2504.19966v3] Quantum circuit lower bounds in the magic hierarchy - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1505.02372v3] On Asymptotic Gate Complexity and Depth of Reversible Circuits With Additional Memory - [http://arxiv.org/abs/1109.4165v2] Quantum Query Complexity of Subgraph Containment with Constant-sized Certificates - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/1709.08890v2] Partial matching width and its application to lower bounds for branching programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def build_dnf(n):
    return [random.choice([0, 1]) for _ in range(2**(n-1))] + [random.randint(0, n) for _ in range(2**(n-1))]
```

```

def build_cnf(n):
    return [[random.choice([0, 1]) for _ in range(n)] for _ in range(2**(n-1))]

def build_block_xor(n):
    block_size = int(n**0.5)
    blocks = [build_dnf(block_size) for _ in range(block_size)]
    result = []
    for i in range(2**(block_size)):
        block_output = 0
        for j in range(block_size):
            if blocks[j][i % block_size] == 1:
                block_output ^= 1
        result.append(block_output)
    return result

def support(gate, assignments):
    return [i for i, bit in enumerate(assignments) if gate[i] == 1]

def build_bipartite_graph(C, assignments):
    G = []
    for g in C:
        G.extend([(g, h) for h in C if set(support(g, assignments)) <= set(support(h, assignments))])
    return G

def hopcroft_karp(G):
    n = len(G)
    U = set(range(n // 2))
    V = set(range(n // 2, n))
    match = [-1] * n
    dist = [0] * n

    def bfs():
        queue = []
        for u in U:
            if match[u] == -1:
                dist[u] = 0
                queue.append(u)
            else:
                dist[u] = float('inf')
        dist[float('inf')] = float('inf')
        while queue:
            u = queue.pop(0)
            if dist[u] < dist[float('inf')]:
                for v in V:
                    if (u, v) in G and dist[v] == float('inf'):
                        dist[v] = dist[u] + 1
                        queue.append(v)
        return dist[float('inf')] != float('inf')

    def dfs(u):
        if u in U:
            for v in V:
                if (u, v) in G and dist[v] == dist[u] + 1:
                    dist[v] = float('inf')
                    if match[v] == -1 or dfs(match[v]):
                        match[u] = v

```

```

        match[v] = u
        return True
    return False
else:
    for (u, v) in G:
        if u == u and dist[v] == dist[u] + 1:
            dist[v] = float('inf')
            if dfs(v):
                return True
    return False

while bfs():
    for u in U:
        if match[u] == -1 and dfs(u):
            pass

return sum(1 for u in U if match[u] != -1)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14]
    results = []

    for n in n_values:
        C_dnf = build_dnf(n)
        C_cnf = build_cnf(n)
        C_block_xor = build_block_xor(n)

        assignments = [random.randint(0, 1) for _ in range(n)]

        for C in [C_dnf, C_cnf, C_block_xor]:
            size_C = len(C)
            w_C = hopcroft_karp(build_bipartite_graph(C, assignments))
            ratio = Fraction(w_C, n).limit_denominator()
            results.append({"n": n, "size": size_C, "w_C": w_C, "ratio": ratio})

    min_ratio = min(result["ratio"] for result in results)
    mean_ratio = sum(result["ratio"] for result in results) / len(results)

    conjecture_holds = min_ratio >= Fraction(2, 5) and mean_ratio >= Fraction(1, 2)
    counterexample = "" if conjecture_holds else "min_ratio < 0.4"

    return {
        "metric_name": "w(C)/log_2(size(C))",
        "metric_value": float(mean_ratio),
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    for seed in seeds:

```

```

    trial = run_trial(seed)
    print(f"TRIAL: {trial}")

ratios = [result["ratio"] for result in results]
min_ratio = min(ratios)
mean_ratio = sum(ratios) / len(ratios)

if all(r >= Fraction(2, 5) for r in ratios):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction=1")
elif any(r < Fraction(2, 5) for r in ratios):
    first_failing_seed = seeds[ratios.index(min(ratios))]
    print(f"RESULT: FALSIFIED counterexample='min_ratio < 0.4' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00eea011.py", line 133, in <module>
    trial = run_trial(seed)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00eea011.py", line 110, in run_trial
    w_C = hopcroft_karp(build_bipartite_graph(C, assignments))
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00eea011.py", line 42, in build_bipartite_graph
    G.extend([(g, h) for h in C if set(support(g, assignments)) <= set(support(h, assignments))])
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_00eea011.py", line 37, in support
    return [i for i, bit in enumerate(assignments) if gate[i] == 1]
            ~~~~~^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in the `support()` function ('int' object is not subscriptable), indicating a bug in how gates are represented or indexed; no instances were evaluated, so neither the support nor falsification condition can be assessed. | next: Fix the gate representation in `support()`: ensure each gate is stored as a truth-table vector (list/array indexable by assignment index) rather than a scalar, then re-run the 360-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	296396
2	preregistration	claude_max	opus	0	0	5882
3	novelty	claude_max	opus	0	0	7295
4	novelty	claude_max	opus	0	0	12135
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14096
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14424
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16547
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14176
9	judge	claude_max	opus	0	0	4830

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 385781 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/f2533eda1d01.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f2533eda1d01.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f2533eda1d01.tar.gz` (if generated)